Project Nexus:
Enterprise Project Discovery Agent

Deep-Research-inspired AI agent

7 March 2026

Use Case

Challenge

- Has something similar already been done?
- What worked / failed?
- What external examples exist?
- What risks should be considered?

Target users

- AI and innovation teams
- Product owners and R&D teams
- Business stakeholders

Goal: reduce repeated research effort and improve reuse of internal knowledge.

Problem & Value

The Problem

- **Knowledge Silos**: Information overload and fragmented manual research.
- **Data Leakage Risks**: Standard AI tools lack physical access control for confidential data.
- **LLM Hallucinations**: Unconstrained agents invent internal cases or hallucinate facts.

The Value

- **Efficiency**: Faster project discovery and structured decision support.
- **Security**: Secure, permission-aware retrieval (Silent Filtering).
- **Reliability**: Traceable research workflow with deterministic citations.

Dataset & Scope

Internal dataset (synthetic)

- 8 mock enterprise initiatives stored in JSON
- Covers representative discovery scenarios rather than large-scale retrieval
- Includes permission labels (C1 / C2) to simulate role-based access behavior

MVP design impact

- Simple local similarity search instead of full RAG
- Focus on **top-1 retrieval** correctness, not top-k ranking
- Evaluation is **scenario-based**: retrieval, RBAC filtering, report structure, and traceability

Example record

- project_id
- Title
- Year
- Team
- Tech_stack
- objective
- outcome
- lessons_learned
- Status
- confidentiality

Takeaway: The MVP validates workflow and enterprise constraints, not large-scale search performance.

Solution Overview

1. Input

- Raw project idea from the user.
- Optional role context (standard / admin)

2. Agentic Engine

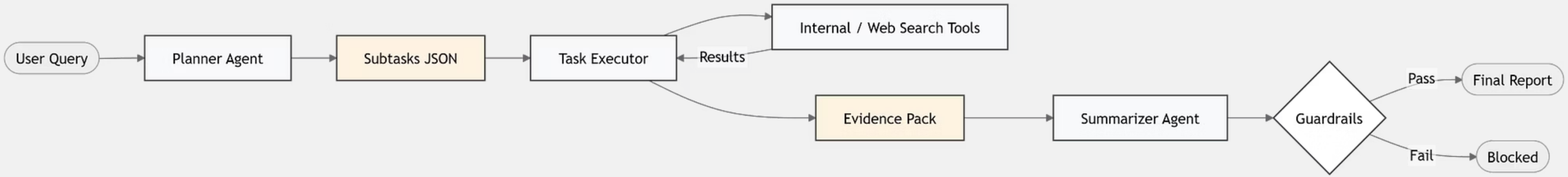
- **Planner**: decomposes the query into research subtasks.
- **Executor**: retrieves internal and external evidence
- **Summarizer**: synthesizes evidence to synthesize findings.
- **Guardrails**: checks structure, citations, and restricted leakage

3. Output

- Similar internal initiatives
- Lessons learned
- External examples
- Risks and recommendation

Workflow

Single-pass agent workflow with explicit state transitions



Evaluation

Success Criteria

- Relevant internal retrieval
- Structured report generation
- No restricted leakage for standard users
- Traceable execution

Current Evaluation

- Happy path: retrieval hit+ report structure
- RBAC checks: standard filtering, admin access
- Edge cases: early reject and keyword-trap scenarios

Runtime Signals

- Internal results count
- Web results count
- Latency
- Warnings

Deployment Plan

Dimension	Current MVP (PoC)	Production Target
Architecture & Hosting	Streamlit + Local Python backend	Containerized microservice + Internal Web Portal Web Portal
Data & Search Layer	Synthetic JSON + Local NumPy similarity	Relational / search store with hybrid retrieval
Security & Auth	Mocked RBAC	Enterprise SSO / RBAC
LLM Infrastructure	Direct API-based LLM access	API gateway or compliant hosted/local model setup
Observability	Local JSON trace logs	Centralized logging and monitoring

Limitations / Next Steps

Limitations

- Synthetic internal data only
- Lightweight retrieval stack
- Minimal guardrails
- Prototype-level access control
- Single-pass workflow

Next Steps

- Hybrid retrieval
- Stronger source handling
- Enterprise integration
- Improved evaluation
- Reflection / recovery loop

Use of AI in Development

How I used AI

- **Strict AI-assist, not vibe coding**
- AI as an assistant, not an autonomous developer
- Architecture, workflow boundaries, and trade-offs stayed human-led

Where AI helped most

- **Velocity and focus**
- Reduced coding friction, so I could spend more time on design and business framing
- Accelerated debugging and just-in-time learning

How I validated the output

- **Zero-trust model**
- Manual review before execution
- Checked alignment with the intended workflow
- Validated through tests and trace inspection